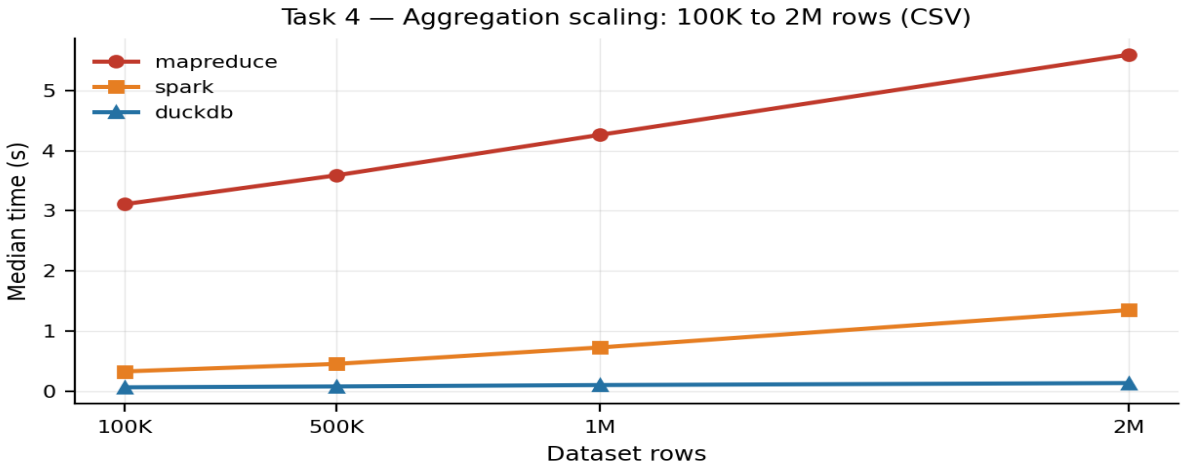
Observations Report — MapReduce vs Spark vs DuckDB

Student ID: MT25063 | Data: NYC Yellow Taxi, Jan 2026 | Machine: AMD Ryzen 7 7840HS, 8C/16T @3.8GHz base, 14GB RAM, 192GB NVMe SSD, Ubuntu 26.04.1 LTS | Python 3.11.11, mrjob 0.7.4, PySpark 4.2.0, DuckDB 1.5.5

1. Part A — Predictions (made before running the timed experiments)

Scenario	Predicted best tool	Reason	Outcome
Aggregate 1M-row CSV by pickup location	DuckDB	Vectorized in-process SQL engine; no cluster/serialization overhead for a single-machine scan+groupby.	Confirmed
Filter 2M-row Parquet, return 4 columns	DuckDB	Columnar Parquet reader should prune both rows (predicate) and columns (projection) before materializing anything.	Confirmed
Join 2M taxi rows with the small zone lookup	Spark	A join is the most “distributed-shaped” operation of the set, so the engine built for distributed joins should lead.	Wrong – see §5
Scale the same aggregation 100K→2M rows	DuckDB	Lowest fixed overhead of the three, so its curve should stay flattest as rows grow.	Confirmed
Repeat the same query several times on the same data	DuckDB	OS-level file caching plus near-zero per-query overhead should beat explicit Spark caching.	Confirmed

2. Task 4 — Scaling plot (median of 3 runs, CSV input)



All three systems grow with rows, but not from the same starting point. MapReduce's curve is nearly flat in relative terms (3.11s→5.59s, a 1.8x increase for 20x more data) because a fixed JVM-less but process-heavy startup/serialization cost dominates at this scale, not per-row work. Spark grows faster in relative terms (0.33s→1.35s, 4.1x) since its fixed SparkSession/task-scheduling overhead is smaller, letting real per-row cost show through sooner. DuckDB stays almost flat in absolute terms (0.068s→0.139s) because its vectorized, in-process scan has negligible fixed cost and CSV parsing this size barely stresses it. The ranking (DuckDB < Spark < MapReduce) holds at every size tested. *Reproducibility check:* an independent rerun on the same machine reproduced the same ranking and shape at every size (DuckDB 0.072s→0.143s, Spark 0.316s→1.125s, MapReduce 3.21s→5.63s across 100K→2M), so this isn't a one-off measurement.

3. Median timings — main comparisons (seconds)

Comparison	MapReduce	Spark	DuckDB
Task 1/4 aggregation @ 1M rows (csv)	4.261	0.730	0.106
Task 2 filter, Query A / all cols @ 2M (parquet)	—	0.224	0.045
Task 2 filter, Query B / 4 cols @ 2M (parquet)	—	0.115	0.013
Task 3 join + top-5 @ 2M (parquet)	— (not implemented)	0.400	0.017
Task 5 repeated query, per-run median @ 1M (parquet)	—	0.154 uncached / 0.136 cached	0.0024

4. Three observations that were surprising

(a) MapReduce's runtime barely moved with data size: a 20x increase in rows (100K→2M) only raised its time 1.8x, so nearly all of its ~3–5.6s is fixed per-job overhead, not row processing.
(b) In Task 5, Spark's very first uncached run (1.265s) was 6–9x slower than every later uncached run on the identical query (0.14–0.19s) — JVM/codegen warm-up cost showed up before caching was even applied.
(c) Building the Spark cache (1.047s, one-time count()) cost more by itself than all five DuckDB repeated runs combined (5 x 0.0024s ≈ 0.012s) — for data this small, DuckDB never needed an explicit cache to already be effectively “cached.”

5. A prediction that was wrong

The join scenario was predicted for Spark, reasoning that a join is the most inherently distributed operation and Spark is purpose-built for distributed joins. In practice DuckDB finished the 2M-row join + top-5 in 0.017–0.019s versus Spark's 0.39–0.40s — roughly 20x faster, confirmed across two independent runs. The lesson: “built for distributed work” only pays off once data is actually distributed or too large for one machine. The plan shows Spark *did* make the algorithmically correct choice (BroadcastHashJoin, no shuffle of the large side), so this isn't Spark picking a bad join strategy — it still loses because a laptop-scale job never needed a distributed engine's coordination machinery in the first place, and DuckDB's in-process hash join has none of that overhead to pay.

6. Projection/filter behavior (Task 2) and data movement/join strategy (Task 3)

Task 2: asking for 4 columns instead of all columns made both engines faster on the identical filtered row set — Spark 0.224s→0.115s (~2x) and DuckDB 0.045s→0.013s (~3.6x). Spark's terminal output confirms both queries matched the same 6,735 rows, so the entire gap comes from bytes read, not different data —

exactly the columnar-pruning behavior predicted.
Task 3 (now confirmed from the actual physical plan): DuckDB beat Spark on the join by ~21x (0.019s vs 0.39s median in the rerun). Spark's plan shows a **BroadcastHashJoin Inner BuildRight**, not a shuffle/sort-merge join: the parquet scan reads only 2 of ~19 columns (`PULocationID, fare_amount`) with `PushedFilters=[IsNotNull(fare_amount), fare_amount>=0, IsNotNull(PULocationID)]` pushed to the scan, and the 265-row zone lookup is broadcast (BroadcastExchange, 32.0 MiB) instead of shuffled. The one remaining Exchange in the plan is a 200-way hash-partition shuffle for the *final* Borough+Zone aggregation, not for the join itself — AQE then coalesces it. So Spark already avoids the expensive shuffle on the large side; the 21x gap is what's left of JVM startup, task scheduling and codegen once the algorithmic choice is optimal on both sides.

Task 3 result — top 5 pickup zones by total fare revenue (2M-row join)

Borough	Zone	Total fare ($)
Queens	JFK Airport	6,849,853.78
Queens	LaGuardia Airport	2,825,841.42
Manhattan	Midtown Center	1,475,111.82
Manhattan	Upper East Side South	1,281,774.03
Manhattan	Upper East Side North	1,223,447.97

7. Task 6 — tool choice by scenario

#	Situation (abridged)	Choice	Justification
A	5GB Parquet, 16GB RAM laptop, 8 SQL queries, data stays local	DuckDB	Fits comfortably in/near RAM; columnar pushdown suits ad-hoc SQL; no cluster needed for this scale.
B	20TB immutable logs already distributed across hundreds of machines; one nightly fault-tolerant summary	Spark	Data already distributed and too large for one machine; Spark gives cluster-scale fault tolerance with lower shuffle/I-O overhead than classic MapReduce for a single batch job.
C	Multi-stage pipeline: repeated joins on large distributed data, transforms, ML training on a cluster	Spark	Iterative/multi-stage workload benefits from Spark's in-memory caching between stages and its DataFrame/ML ecosystem across a cluster — MapReduce has no efficient iteration model.
D	30GB CSV, 32GB RAM workstation, fast NVMe, 5 simple aggregates, no cluster available	DuckDB	Single machine, no cluster; dataset is close to RAM-sized and NVMe is fast, so an in-process columnar/CSV engine avoids cluster overhead entirely.
E	2GB dataset fits on one laptop; colleague proposes a Spark cluster “because Big Data”	DuckDB	Classic overkill case: 2GB fits trivially in memory, so a cluster only adds startup/coordination overhead for zero benefit — this experiment's own numbers show Spark losing to DuckDB even at 2M rows / ~200MB.
F	200GB dataset; no info on format, query type, memory, storage, or distribution	Insufficient information	The right choice hinges on exactly the unknowns here (format, single vs. distributed storage, query pattern, available RAM) — picking a tool without them is a guess, not reasoning.

8. Final conclusion (≤150 words) — what mattered more than dataset size

Across every task, fixed per-job overhead — not raw row count — decided the ranking. MapReduce's near-flat curve across a 20x size increase, Spark's costly first-run JIT/codegen warm-up in Task 5, and the one-time cache-build cost exceeding DuckDB's entire five-run repeated workload all point the same way: whether an engine pays startup, JVM, serialization or coordination cost before doing any real work mattered far more than how many rows it eventually touched, at least at the 100K–2M row scale tested here. Whether data was already distributed also mattered independently of size — the join task looked like Spark's home turf on paper, but with a small lookup table and everything on one laptop, DuckDB's zero-coordination in-process execution won decisively anyway. The practical rule this suggests: match the tool to where the data already lives and how often the job restarts, not just to how large the data is.